UTEQ

Ing. Desarrollo de software multiplataforma

Grupo IDGS15

Proyecto Integrador: Pipeline CI/CD Automatizado para API REST con Docker, GitHub Actions y AWS EC2

Gestión del Proceso de Desarrollo de Software

Unidad I - Introducción a DevOps

Parcial 1

Integrante

Damian Anwar Mata Nevarez

Matrícula: 2023371173

Docente

EMMANUEL MARTINEZ HERNANDEZ

Fecha: 7 de octubre de 2026

Índice

Contenido	Página
1. Introducción	4
2. Arquitectura de la solución	5
3. Desarrollo y aseguramiento de la calidad	6
3.1. Pruebas automatizadas y umbral de cobertura	6
3.2. Contenedorización con Docker	7
4. Integración continua con GitHub Actions	8
5. Despliegue continuo en AWS EC2	9
5.1. Infraestructura y seguridad	9
5.2. El trabajo de despliegue	10
6. Resultados y verificación	11
6.1. Verificación end-to-end del despliegue	11
6.2. Demostración práctica en vivo	12
6.3. Entregables	12
7. Conclusiones	13
8. Fuentes de información	14

Este reporte documenta la implementación y la verificación local. Las etapas que requieren cuentas de GitHub, Docker Hub y una instancia AWS EC2 se describen con su estado pendiente de ejecución externa.

Índice de figuras

Figura	Descripción	Página
1	Arquitectura prevista del pipeline CI/CD	5
2	Cobertura real de Jest y umbral mínimo del 70%	6
3	Extracto del Dockerfile implementado	7
4	Etapas y dependencias del workflow	8
5	Secuencia azul-verde y comprobación del commit	10
6	Registro de verificación Docker local	11

Las figuras son esquemas propios, extractos de archivos del proyecto o resultados locales existentes. Cada pie identifica su procedencia. No se incluyen fotografías del pizarrón ni evidencias del proyecto de otro alumno.

1. Introducción

Una API REST necesita ofrecer respuestas correctas y contar con un procedimiento reproducible para entregar cambios. Instalar dependencias, ejecutar pruebas y reemplazar una versión manualmente puede introducir diferencias entre ambientes y dificultar el seguimiento de errores. El proyecto integrador aborda este problema mediante la preparación de un pipeline de integración y despliegue continuos para una API desarrollada con Node.js, Express y SQLite.

DevOps relaciona el desarrollo con la operación del servicio mediante prácticas de colaboración, automatización y verificación. La integración continua comprueba las modificaciones del código antes de publicarlas: instala las dependencias del archivo de bloqueo, ejecuta las pruebas y evalúa la cobertura. El despliegue continuo utiliza el resultado aprobado para construir una imagen y actualizar el servidor. Estas etapas permiten identificar el punto en que una modificación falla y conservar trazabilidad entre código e imagen.

El objetivo de este trabajo es preparar un flujo que ejecute pruebas con una cobertura mínima del 70%, empaquete la aplicación en Docker, publique imágenes en Docker Hub y despliegue el commit correspondiente en una instancia Ubuntu de AWS EC2. La actualización se diseña con dos contenedores detrás de Nginx para verificar la candidata antes de transferirle el tráfico. La disponibilidad durante esa transición deberá medirse cuando exista una instancia operativa.

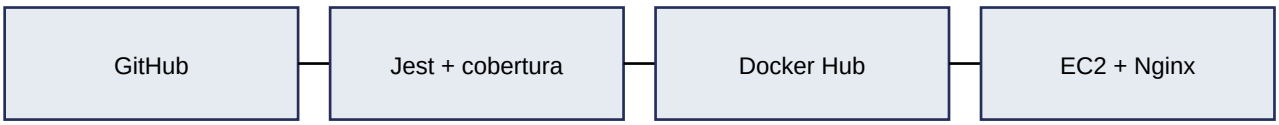
La API tiene exactamente seis endpoints: cinco operaciones de creación, listado, consulta, reemplazo y eliminación de usuarios, y una consulta de salud. SQLite conserva los registros; los esquemas validan los datos y las consultas parametrizadas separan las entradas del código SQL. El endpoint de salud también devuelve el commit, por lo que puede utilizarse para verificar una actualización sin agregar otra ruta.

Las herramientas utilizadas son Jest y Supertest para evaluar el comportamiento HTTP, Docker para construir el ambiente de ejecución, GitHub Actions para coordinar las etapas y scripts Bash para preparar y actualizar EC2. Los secretos de acceso se reciben desde la configuración externa del repositorio. El trabajo se realiza en una copia independiente ubicada en la carpeta de décimo cuatrimestre.

El reporte presenta primero la arquitectura, después el aseguramiento de calidad y la contenedorización, y a continuación las etapas de CI y CD. La sección de resultados distingue las 29 pruebas aprobadas y la verificación Docker local de las actividades externas pendientes. Finalmente se incluyen la demostración propuesta, los entregables, las conclusiones y las fuentes. Los resultados del compañero que sirvió como referencia de estructura no se atribuyen a este proyecto.

2. Arquitectura de la solución

El flujo conecta el repositorio con la aplicación desplegada. Los pull requests a main ejecutan la validación. Un push a main habilita, después de las pruebas, la publicación de la imagen y el trabajo de despliegue.



main > test > build-and-push > deploy-ec2

Figura 1. Arquitectura prevista. Elaboración propia a partir del workflow implementado; ejecución externa pendiente.

```
backend-api-integrador/
  .github/workflows/main.yml   Pipeline de tres trabajos
  index.js / schemas.js        Rutas y validación
  database.js                  Persistencia SQLite
  index.test.js                Pruebas HTTP y de errores
  jest.config.js               Cobertura mínima del 70%
  Dockerfile / .dockerignore   Imagen y contexto de build
  compose.yml                  Puerto 80 y volúmenes
  scripts/                     Despliegue y verificaciones
  docs/ / report/              Guías y resultados
  README.md                    Configuración y ejecución
```

Método	Ruta	Función
POST	/api/users	Crear usuario
GET	/api/users	Listar usuarios
GET	/api/users/:id	Consultar usuario
PUT	/api/users/:id	Reemplazar usuario
DELETE	/api/users/:id	Eliminar usuario
GET	/api/health	Comprobar salud y commit

El nombre y el correo son obligatorios. Se rechazan correos duplicados, identificadores inválidos y propiedades adicionales. La salud consulta SQLite; la base se almacena fuera de la capa efímera del contenedor. Las rutas que no pertenecen a este inventario devuelven 404.

3. Desarrollo y aseguramiento de la calidad

3.1. Pruebas automatizadas y umbral de cobertura

Jest ejecuta las pruebas y Supertest envía solicitudes a Express. La base de pruebas se crea en memoria y no modifica los datos del proyecto. La configuración incluye index.js, database.js y schemas.js en la cobertura y establece un mínimo global del 70% para sentencias, ramas, funciones y líneas.

```
npm test -- --verbose --json --outputFile=report/test-results.json
node scripts/coverage-summary.js
```

El resultado registrado es de 29 pruebas aprobadas en 1 suite. Se evalúan el CRUD, persistencia entre solicitudes, validación, correos únicos, errores de SQLite, JSON inválido y salud. Una prueba cuenta las seis rutas; otras comprueban la ausencia de endpoints adicionales.

Métrica	Resultado	Umbral
Líneas	100%	70%
Sentencias	96.15%	70%
Funciones	95.83%	70%
Ramas	88.37%	70%



Figura 2. Cobertura real del proyecto. Fuente: coverage/coverage-summary.json. La línea roja indica el 70%.

También se verifican cuatro escenarios simulados del despliegue: candidata sana, candidata no sana, configuración Nginx inválida y commit incorrecto. Son comprobaciones del script con herramientas simuladas; su resultado no acredita un despliegue en AWS.

3.2. Contenedorización con Docker

La imagen utiliza Node.js 22 sobre Debian bookworm-slim en dos etapas. La primera instala las dependencias de producción con npm ci y las herramientas para compilar SQLite. La etapa final recibe las dependencias y el código necesario, ejecuta con el usuario node y expone el puerto interno 3000.

```
FROM node:22-bookworm-slim AS dependencies
WORKDIR /app
# Instalación de herramientas de compilación de SQLite
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

FROM node:22-bookworm-slim
ARG COMMIT_SHA=local
ENV COMMIT_SHA=$COMMIT_SHA
ENV NODE_ENV=production PORT=3000
ENV DB_PATH=/app/storage/data.db
WORKDIR /app
COPY --from=dependencies /app/node_modules ./node_modules
COPY package*.json index.js database.js schemas.js ./
# Creación de directorios y permisos para node
USER node
EXPOSE 3000
# HEALTHCHECK consulta /api/health
CMD ["node", "index.js"]
```

Figura 3. Extracto abreviado del Dockerfile real. Los comentarios sintetizan instrucciones; el archivo completo se conserva en el proyecto.

Docker Compose publica el puerto 80 del equipo hacia el 3000 del contenedor, utiliza restart unless-stopped y conserva la base en un volumen. El HEALTHCHECK consulta /api/health y falla si no recibe HTTP 200. El archivo .dockerignore reduce el contexto y evita incorporar dependencias locales, reportes, bases y secretos a la imagen.

```
docker build -t backend-api-integrador:local .
docker compose up -d --build
curl http://localhost/api/health
```

La construcción y las solicitudes HTTP se verificaron localmente con una publicación temporal en localhost. La prueba también comprobó que un usuario persiste tras reiniciar el contenedor. Este ensayo no constituye una comprobación del puerto 80 público en EC2.

4. Integración continua con GitHub Actions

El archivo .github/workflows/main.yml define tres trabajos encadenados. El trabajo test se ejecuta en push y pull_request sobre main. build-and-push y deploy-ec2 solo se habilitan en un push a main y dependen del éxito del trabajo anterior.

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  test:
    # Node 22, npm ci, Jest, simulaciones y artefactos
  build-and-push:
    needs: test
    # Solo push a main: Buildx, login y publicación
  deploy-ec2:
    needs: build-and-push
    environment: production
    # Solo push a main: SSH y script de despliegue
```

Figura 4. Extracto de etapas y dependencias del workflow. Es configuración implementada, no una captura de GitHub Actions ejecutado.

Validación y artefactos

test configura Node.js 22, instala con npm ci, lista las pruebas, ejecuta Jest y el script de escenarios de despliegue. coverage-summary.js genera el resumen y comprueba el umbral. Los resultados JSON, la cobertura y validation.md se conservan como artefactos para revisar la ejecución.

Construcción y publicación

build-and-push inicia Buildx y utiliza DOCKERHUB_USERNAME y DOCKERHUB_TOKEN para autenticarse. La imagen linux/amd64 se publica como backend-api:latest y backend-api:SHA. El SHA también se inyecta mediante COMMIT_SHA, lo que permite comparar la imagen solicitada con el commit devuelto por la API.

El código de automatización está preparado. La publicación real requiere configurar un repositorio y sus secretos; no se dispone de un enlace de ejecución ni de etiquetas remotas verificadas para este proyecto. Los archivos de resultados locales no sustituyen los logs del runner de GitHub.

5. Despliegue continuo en AWS EC2

5.1. Infraestructura y seguridad

El diseño utiliza una instancia Ubuntu con Docker. Nginx publica HTTP por el puerto 80 y dirige las solicitudes hacia uno de dos contenedores de la API, llamados api-blue y api-green. La base SQLite reside en almacenamiento persistente compartido. La instancia y su disponibilidad todavía deben verificarse en la cuenta del propietario.

El script scripts/setup-ec2.sh prepara las dependencias del servidor. El Security Group debe permitir HTTP en el puerto 80 y SSH en el 22 desde los orígenes autorizados que necesite el mecanismo de despliegue. La apertura y conectividad de esos puertos requieren una comprobación en la infraestructura real.

Secreto	Uso
DOCKERHUB_USERNAME	Usuario del registro
DOCKERHUB_TOKEN	Autenticación en Docker Hub
EC2_HOST	Host de la instancia
EC2_USERNAME o EC2_USER	Usuario SSH
EC2_SSH_KEY	Clave privada de acceso
EC2_KNOWN_HOSTS	Clave del host SSH verificada

El workflow recibe las credenciales mediante GitHub Secrets, crea archivos SSH temporales con permisos restrictivos y exige StrictHostKeyChecking=yes. El job elimina la clave al finalizar. El token de Docker se transmite por entrada estándar. El reporte no incorpora valores reales de credenciales ni reutiliza los del documento de referencia.

La configuración production y el grupo de concurrencia ec2-production serializan las actualizaciones. Antes de completar esta etapa deben configurarse los secretos, la verificación del host y el servidor. No se declara una IP o URL pública mientras no exista una comprobación propia.

5.2. El trabajo de despliegue

deploy-ec2 recibe la imagen etiquetada por el SHA del commit. Se autentica en el registro, copia scripts/deploy.sh al servidor y lo ejecuta por SSH. El script bloquea actualizaciones concurrentes, inicia la candidata y verifica su salud antes de cambiar Nginx.

```
1. Adquirir bloqueo de despliegue.
2. Descargar backend-api:<SHA>.
3. Identificar el contenedor activo y arrancar el alternativo.
4. Consultar /api/health en la candidata.
5. Validar la configuración Nginx y recargarla.
6. Consultar el servicio y comparar su commit con <SHA>.
7. Esperar el período de drenaje de 30 segundos.
8. Detener la versión anterior tras la comprobación.

Ante fallo: conservar o restaurar el tráfico anterior.
```

Figura 5. Secuencia azul-verde implementada en scripts/deploy.sh. Elaboración propia; pendiente de ensayo en EC2.

La verificación del commit protege contra una respuesta saludable de una versión equivocada. La base utiliza WAL y un tiempo de espera de cinco segundos ante bloqueos. Durante la transición, los cambios de esquema futuros deberán ser compatibles con ambas versiones.

Escenario simulado	Criterio evaluado
Candidata sana	Cambio de tráfico antes de detener la anterior
Candidata no sana	No retirar la versión operativa
Nginx inválido	Evitar aplicar la configuración defectuosa
Commit incorrecto	Rechazar la versión y restaurar el tráfico

Los cuatro escenarios del script fueron aprobados en simulación local. La primera instalación carece de una versión anterior y una sola EC2 no cubre una caída de la instancia. La continuidad de las actualizaciones exige mediciones reales; el diseño por sí solo no demuestra ausencia de interrupciones.

6. Resultados y verificación

6.1. Verificación end-to-end del despliegue

La comprobación disponible cubre la aplicación y el contenedor en el equipo local: construcción de imagen, salud, operaciones de usuarios, persistencia tras reinicio y rechazo de una ruta adicional. El registro del ensayo identifica una imagen local y seis endpoints.


```
{
  "date": "2026-10-07",
  "image": "backend-api-integrador:local",
  "endpoints": 6,
  "build": true,
  "health": true,
  "usersCrud": true,
  "restartPersistence": true,
  "extraRouteAbsent": true,
  "commit": "local"
}
```


Figura 6. Registro real del ensayo Docker del 7 de octubre de 2026. Fuente: report/docker-validation.json; alcance local.

Comprobación	Resultado
Pruebas de aplicación (29)	Aprobadas
Cobertura global	Las cuatro métricas superan el 70%
Imagen, CRUD y persistencia en Docker	Verificados localmente
Simulación del script de despliegue	Cuatro escenarios aprobados
Publicación en GitHub y Docker Hub	Pendiente
API pública y actualización en EC2	Pendiente

La verificación end-to-end externa deberá enlazar un push con la ejecución de los tres trabajos, la etiqueta remota del SHA y una respuesta HTTP de la API pública que devuelva ese mismo commit. Todavía no existe evidencia propia de esa cadena completa. Docker Desktop presentó además un fallo recurrente al reiniciar, recuperado mediante un procedimiento temporal local; esto no valida ni invalida la ejecución futura en Ubuntu.

6.2. Demostración práctica en vivo

La demostración propuesta conserva los seis endpoints. Se modificará el mensaje devuelto por /api/health, se ejecutarán las pruebas y se hará push a main. Se observarán los tres trabajos y se comparará el commit de la API pública con el SHA publicado en Docker Hub.

```
npm test -- --verbose
git add index.js
git commit -m "Actualiza mensaje de salud"
git push origin main

# En otra terminal, con la URL pública configurada:
node scripts/check-continuity.js
```

check-continuity.js utiliza API_BASE_URL y MONITOR_SECONDS, consulta /api/health periódicamente y guarda códigos, salud y latencia en report/continuity.json. El resultado deberá analizarse junto con el intervalo de despliegue. No se asigna un tiempo de actualización ni un porcentaje de continuidad antes de medirlos.

Las evidencias por incorporar son el panel de Actions con los trabajos concluidos, la etiqueta del commit en Docker Hub, la respuesta pública antes y después del cambio y el registro de solicitudes durante la actualización. Deben mostrar el proyecto propio y evitar exponer secretos. La demostración descrita en esta sección está pendiente de ejecución externa.

6.3. Entregables

Entregable	Ubicación o estado
Código, pruebas y configuración	backend-api-integrador/
Workflow	.github/workflows/main.yml
Contenedorización	Dockerfile, .dockerignore y compose.yml
Guías	README.md, docs/ y GUIA_PRUEBAS.md
Resultados locales	report/test-results.json, validation.md y docker-validation.json
Cobertura HTML	coverage/lcov-report/index.html
Reporte técnico	Reporte_integrador.pdf
Repositorio, imagen remota y URL pública	Pendientes de publicación y verificación

7. Conclusiones

El proyecto cuenta con una API de exactamente seis endpoints, validación de entradas, persistencia en SQLite y una base de pruebas reproducible. Las 29 pruebas aprobadas y la cobertura registrada respaldan los comportamientos evaluados. El umbral del 70% se incorpora a la configuración para impedir que un cambio aprobado por el flujo reduzca cualquiera de las cuatro métricas por debajo del mínimo.

La contenedorización permitió construir y comprobar la aplicación con sus dependencias de producción en un ambiente definido. El ensayo local verificó las operaciones de usuarios, la salud y la conservación de registros tras el reinicio del contenedor. El uso de un usuario sin privilegios y de almacenamiento persistente forma parte de la configuración de ejecución. La recuperación temporal de Docker Desktop también mostró la necesidad de distinguir problemas del equipo de desarrollo de resultados propios de la API.

El workflow preparado organiza la entrega en tres responsabilidades: pruebas, construcción y publicación, y despliegue. Las etiquetas por commit y el valor devuelto por /api/health permiten rastrear qué código debería atender solicitudes. La gestión de secretos y la verificación del host SSH mantienen las credenciales fuera del código y proporcionan una base para configurar la conexión con el servidor.

La estrategia azul-verde comprueba la candidata y el commit antes de retirar la versión anterior. Los cuatro escenarios simulados respaldan la secuencia prevista y el comportamiento ante fallos evaluados. Sin embargo, aún hace falta demostrar ese proceso en EC2 y medir las solicitudes durante una actualización para establecer su continuidad efectiva.

El cierre de la entrega requiere publicar el repositorio, configurar Docker Hub y los secretos, preparar EC2 y ejecutar la demostración externa. Como mejoras posteriores pueden incorporarse análisis estático, revisión de dependencias y un entorno de pruebas previo a producción. Este reporte conserva el orden académico de la referencia y presenta únicamente las características y evidencias de la implementación propia.

8. Fuentes de información

1. GitHub. Documentación de GitHub Actions. Configuración y sintaxis de workflows. https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

2. Docker. Multi-stage builds. Construcción de imágenes en varias etapas. https://docs.docker.com/build/building/multi-stage/

3. Amazon Web Services. Security groups for your Amazon EC2 instances. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html

4. Jest. Configuración: coverageThreshold y coverageReporters. https://jestjs.io/docs/configuration

5. SQLite. Write-Ahead Logging. Descripción del modo WAL. https://www.sqlite.org/wal.html

6. Documento de la asignatura. Proyecto Integrador: Pipeline CI/CD Automatizado para API REST con Docker, GitHub Actions y AWS EC2, parcial 1. Material proporcionado por el alumno; alcance aclarado a seis endpoints.

7. Reporte de referencia proporcionado por el alumno: reporte_pipeline.pdf. Utilizado exclusivamente para adaptar la organización académica: índices, ocho secciones y subdivisiones de calidad, infraestructura y resultados.

8. Evidencias propias: report/test-results.json, coverage/coverage-summary.json, report/docker-validation.json, .github/workflows/main.yml y scripts/test-deploy.sh. Resultados y configuración de backend-api-integrador.

Documentación de referencia identificada el 7 de octubre de 2026. Los datos institucionales y el logotipo fueron proporcionados por Damian Anwar Mata Nevarez. La numeración de páginas de los índices corresponde a este reporte.